Section 1: Kartik

- **Role:** Model Training & Final Architecture Implementation Lead.
- **Objective:** Execute the large-scale Kaggle training of ConvFormer and produce the final validated model weights for deployment.
- **Exact Responsibility:** Construct the PyTorch ConvFormer architecture, configure the multi-year Kaggle training environment, enforce the chronological train/validation split, compute train-only normalization statistics, manage the training loop (AdamW, ReduceLROnPlateau), and export the final model state.
- **Why this responsibility exists:** The neural network weights are the core intellectual property of OceanEmbed; training requires dedicated iteration on hyperparameters and memory optimization distinct from data engineering.
- **Inputs Received:** Chunked multi-year Zarr/NetCDF datasets (1993–2020) and chronological split indices.
- **Exact Input Format:** PyTorch DataLoader yielding (B, 5, 7, 101, 241) input tensors and (B, 5, 15, 101, 241) target tensors.
- **Expected Outputs:** The frozen neural network, architecture configuration, and the normalization statistics required for inference.
- **Exact Output Format:** final_model.pt (PyTorch state dict), model_config.json, and normalization_stats_train.json.
- **Files/Directories Owned:** src/models/*, src/training/*, experiments/*.
- **Files Allowed to Modify:** Model definitions, training scripts, experiment configurations.
- **Files Must Not Modify:** Preprocessing logic (src/preprocessing/*), evaluation metrics, backend API routes.
- **Dependencies on Other Members:** Requires the completed, leak-free Zarr dataset from Member 1 and the formal PASS certification from Member 2.
- **What Others Depend On:** Member 3 requires the .pt file and normalization JSON to build the API. Member 5 requires the model outputs for evaluation.
- **Detailed Tasks:**
  - Implement the ConvFormer layers (CNN feature extractor, ConvLSTM, Patch Embedding, Transformer Encoder, Decoder, 1x1 Depth Projection).
  - Write the custom PyTorch Dataset class to lazy-load temporal windows from Zarr chunks.
  - Compute global mean and standard deviation strictly on the 1993–2018 training split.
  - Execute the Kaggle training run with masked MSE loss (evaluating only where ~torch.isnan(target) is True).
  - Implement early stopping using the 2019–2020 validation split.
- **Step-by-Step Implementation Plan:**
1. Draft unit tests for the ConvFormer forward pass with dummy tensors.
2. Write the sequence generator to create T=5 windows.
3. Run a 1-epoch sanity check locally to verify the loss decreases.
4. Migrate the codebase to Kaggle, configure GPU accelerators, and mount the dataset.
5. Run the full training loop and monitor validation loss.

6. Export the best checkpoint.

- **Expected Code/Components:** convformer.py, dataset.py, train.py, config.py.
- **Expected Tests:** Forward pass tensor shape validation, zero-gradient checks on target NaNs, batch-independence tests.
- **Validation Criteria:** Training loss converges; validation loss stabilizes without overfitting; the model outputs the exact (B, 5, 15, 101, 241) shape without NaNs.
- **Definition of Done:** final_model.pt is exported and successfully loaded by Member 3's backend script.
- **Deliverables:** The model weights, configuration, and normalization statistics.
- **Handoff Procedure:** Upload the three artifact files to the team's shared repository/storage under a versioned release (e.g., v1.0.0-convformer).
- **Failure Cases:** Out-of-memory (OOM) on Kaggle (mitigation: reduce batch size or enable gradient accumulation); NaNs in loss (mitigation: verify input masking with Member 2).
- **How to Verify Independently:** Load final_model.pt on a local CPU, pass a tensor of torch.ones((1, 5, 7, 101, 241)), and verify it outputs a tensor of (1, 5, 15, 101, 241) without throwing exceptions.
- **Exact Interface Contract:** Consumes (B, 5, 7, 101, 241) float32 tensors. Returns (B, 5, 15, 101, 241) float32 tensors.

Section 2: Member 1

- **Role:** Large-Scale Data Pipeline & Engineering Lead.
- **Objective:** Produce the final multi-year (1993–2023) training, validation, and test datasets in a cloud-optimized format.
- **Exact Responsibility:** Automate the acquisition of raw NetCDFs (SST, SSS, SLA, Currents, ERA5, GLORYS), execute spatial regridding to 0.25°, perform 15-depth vertical interpolation including the 0m extrapolation, apply the ocean mask, and export chunked Zarr arrays.
- **Why this responsibility exists:** 30 years of daily, high-resolution oceanographic data cannot be processed manually or held in RAM; it requires a robust, reproducible ETL pipeline.
- **Inputs Received:** Raw Copernicus (GLORYS, SLA, Currents), OSTIA, SMAP/SMOS, and ERA5 API credentials.
- **Exact Input Format:** Heterogeneous raw NetCDF files with varying spatial resolutions (0.05° to 0.25°) and grid types (cell-centered vs. node-centered).
- **Expected Outputs:** A unified, temporally aligned, spatially uniform dataset.
- **Exact Output Format:** Chunked Zarr stores with shapes (Time, 7, 101, 241) for inputs and (Time, 15, 101, 241) for targets.
- **Files/Directories Owned:** src/preprocessing/*.
- **Files Allowed to Modify:** Ingestion scripts, regridding logic, masking utilities.
- **Files Must Not Modify:** Model training scripts, testing scripts, backend routing.
- **Dependencies on Other Members:** None for initial development.
- **What Others Depend On:** Kartik strictly requires this data to begin Kaggle training.

Member 2 requires it for scientific QA.

- **Detailed Tasks:**
  - Script CDS API calls to download ERA5 U10/V10.
  - Squeeze the redundant depth=1 axis from the SSS data.
  - Convert SST from Kelvin to Celsius.
  - Interpolate offset grids (SLA, Currents) to the exact 0.25° target grid.
  - Implement the 0m linear extrapolation formula for the GLORYS target.
  - Stack the 7 variables chronologically.
- **Step-by-Step Implementation Plan:**

1. Write downloading scripts for all 6 raw sources.
2. Implement xarray-based spatial interpolation functions.
3. Construct the 101x241 master ocean mask.
4. Process 1 month locally to verify alignment.
5. Deploy the script to Kaggle to process the full 1993–2023 period in parallel chunks.
6. Serialize to Zarr.

- **Expected Code/Components:** download.py, regrid.py, vertical_interp.py, build_zarr.py.
- **Expected Tests:** Verify spatial coordinates are exactly 5.0, 5.25... 30.0.
- **Validation Criteria:** All 7 input variables and 15 target depths exist on the identical 101x241 grid for every day from Jan 1, 1993, to Dec 31, 2023.
- **Definition of Done:** Zarr stores are successfully mounted and readable in a Kaggle notebook.
- **Deliverables:** The Zarr dataset and the reproducible preprocessing pipeline script.
- **Handoff Procedure:** Provide the dataset path/URL to Kartik and Member 2, alongside a schema markdown file.
- **Failure Cases:** Missing temporal days in raw data (mitigation: implement linear temporal interpolation for gaps < 2 days, or drop sequence).
- **How to Verify Independently:** Open the Zarr store using xarray, plot the 0m target for a random day, and visually confirm the landmass is masked out (NaN).
- **Exact Interface Contract:** Outputs (T, 7, 101, 241) and (T, 15, 101, 241). Land in targets = NaN. No NaNs in valid ocean regions.

Section 3: Member 2

- **Role:** Dataset Validation & Scientific Quality Assurance.
- **Objective:** Guarantee the multi-year dataset contains zero leakage, physical anomalies, or spatial misalignments before training begins.
- **Exact Responsibility:** Audit the Zarr stores produced by Member 1. Write automated test suites to verify temporal splits, boundary integrity, coordinate alignment, and correct NaN preservation.
- **Why this responsibility exists:** Neural networks will silently learn from flawed data. A dedicated QA role prevents weeks of wasted training time due to preprocessing bugs (e.g., the ASCAT spatial smearing issue).
- **Inputs Received:** Zarr dataset paths from Member 1.
- **Exact Input Format:** (Time, 7, 101, 241) and (Time, 15, 101, 241) Zarr arrays.

- **Expected Outputs:** A formal validation report and an automated test suite.
- **Exact Output Format:** pytest log outputs and a validation_report.md.
- **Files/Directories Owned:** src/preprocessing/validation.py, tests/test_data.py.
- **Files Allowed to Modify:** Test scripts only.
- **Files Must Not Modify:** The actual preprocessing logic or the model architecture.
- **Dependencies on Other Members:** Requires the dataset from Member 1.
- **What Others Depend On:** Kartik cannot begin Kaggle training until Member 2 issues a PASS certification.
- **Detailed Tasks:**
  - Verify chronological splitting (no overlap between 1993-2018, 2019-2020, and 2021-2023).
  - Assert that no target NaNs (land/seafloor) were globally imputed.
  - Verify the ERA5 U10/V10 channels contain 0% missing data.
  - Check physical bounds (e.g., SST is within valid Celsius ranges, not Kelvin).
- **Step-by-Step Implementation Plan:**
1. Write bounds-checking functions for all 7 input variables.
2. Write NaN detection functions to ensure the spatial mask is identical across all timesteps.
3. Execute the test suite against Member 1's 1-month test batch.
4. Execute the test suite against the full 30-year Zarr store.
5. Compile the audit report.
- **Expected Code/Components:** test_bounds.py, test_leakage.py, test_masking.py.
- **Expected Tests:** assert not np.any(np.isnan(era5_data)), assert sst.max() < 40.0.
- **Validation Criteria:** All defined pytest functions return a PASS status.
- **Definition of Done:** The validation_report.md is merged into the repository with all checks passing.
- **Deliverables:** The QA report and the CI/CD-ready test scripts.
- **Handoff Procedure:** Notify Kartik via the team communication channel that the dataset is mathematically and scientifically safe for training.
- **Failure Cases:** Tests uncover target leakage or coordinate mismatch (mitigation: reject the dataset and hand it back to Member 1 with the exact failure logs).
- **How to Verify Independently:** Anyone can run pytest tests/test_data.py on their local machine and see the green PASS outputs.
- **Exact Interface Contract:** Consumes the Zarr schema. Produces Boolean PASS/FAIL signals for the pipeline.

Section 4: Member 3

- **Role:** Backend API & Inference Pipeline Lead.
- **Objective:** Serve the trained ConvFormer model as a robust REST API for the frontend dashboard.
- **Exact Responsibility:** Develop a FastAPI server that ingests new daily NetCDF observations, applies the frozen normalization statistics, constructs the sequence window, executes the PyTorch forward pass, and formats the output into JSON.

- **Why this responsibility exists:** The web dashboard cannot run a PyTorch model directly. A dedicated backend abstracts the neural network into a simple web service.
- **Inputs Received:** final_model.pt, model_config.json, normalization_stats_train.json (from Kartik), and new daily inference requests (from Member 4).
- **Exact Input Format:** HTTP GET/POST requests containing geographic coordinates or bounding boxes.
- **Expected Outputs:** A running web server returning reconstructed temperature data.
- **Exact Output Format:** JSON payloads. Example: { "depths": [0, 5, ..., 1000], "temperatures": [28.1, 27.9, ...] }.
- **Files/Directories Owned:** api/*, src/inference/*.
- **Files Allowed to Modify:** FastAPI routes, inference scripts, Pydantic schemas.
- **Files Must Not Modify:** The core model architecture (src/models/*) or training logic.
- **Dependencies on Other Members:** Requires the .pt file and normalization JSON from Kartik. Requires D26 derivation logic from Member 5.
- **What Others Depend On:** Member 4 (Frontend) strictly depends on this API to build the user interface.
- **Detailed Tasks:**
  - Load the ConvFormer state dictionary into VRAM/RAM on server startup.
  - Implement the /predict/profile endpoint for point-based vertical profiles.
  - Implement the /predict/slice endpoint for horizontal depth mapping.
  - Apply Kartik's normalization_stats_train.json to incoming raw inputs before passing them to the model.
  - Explicitly fill input NaNs (land) with 0.0 after normalization.
- **Step-by-Step Implementation Plan:**

1. Define the Pydantic request and response schemas.
2. Write a dummy endpoint that returns static data for Member 4 to start building against immediately.
3. Integrate the PyTorch model loading mechanism.
4. Write the pre-inference processing function (normalization + NaN filling).
5. Connect the PyTorch output to the JSON response formatter.
6. Deploy locally via Uvicorn.

- **Expected Code/Components:** main.py, routers/predict.py, schemas.py, inference_engine.py.
- **Expected Tests:** Endpoint unit tests using FastAPI.testclient to verify HTTP 200 responses and correct JSON structures.
- **Validation Criteria:** The API successfully returns a 15-depth temperature profile for any valid coordinate in the 5°N–30°N, 45°E–105°E domain within 2 seconds.
- **Definition of Done:** The API is stable, handles OOM gracefully, and is documented via Swagger/OpenAPI.
- **Deliverables:** The FastAPI application and OpenAPI documentation.
- **Handoff Procedure:** Provide the local host URL or deployment endpoint and the Swagger documentation link to Member 4.
- **Failure Cases:** User requests coordinates outside the domain (mitigation: return HTTP

400 Bad Request with a clear error message).
- **How to Verify Independently:** Navigate to http://localhost:8000/docs, execute a test request through the Swagger UI, and inspect the returned JSON.
- **Exact Interface Contract:** Receives raw surface variables. Applies $x_{norm} = (x - \mu) / \sigma$. Returns un-normalized 15-depth Celsius predictions.

Section 5: Member 4

- **Role:** Frontend Web Dashboard & Visualization Lead.
- **Objective:** Build the interactive, user-facing geographic application for OceanEmbed.
- **Exact Responsibility:** Develop a React/Leaflet dashboard that allows users to view horizontal temperature maps, select specific depth levels via a slider, and click coordinates to view vertical temperature profiles and the D26 isotherm depth.
- **Why this responsibility exists:** The final SIH hackathon deliverable requires a functional software product for end-users, not just backend code or PyTorch weights.
- **Inputs Received:** JSON payloads from the backend API (Member 3).
- **Exact Input Format:** JSON arrays of coordinates, depths, and temperatures.
- **Expected Outputs:** A deployed, interactive web application.
- **Exact Output Format:** A React Single Page Application (SPA).
- **Files/Directories Owned:** frontend/* (components, styles, map utilities).
- **Files Allowed to Modify:** All frontend code.
- **Files Must Not Modify:** Backend API routes, inference logic.
- **Dependencies on Other Members:** Requires the API schema and a running backend (or dummy endpoints) from Member 3.
- **What Others Depend On:** The entire team depends on this UI for the final demonstration and presentation.
- **Detailed Tasks:**
  - Initialize the React application and integrate Leaflet/React-Leaflet.
  - Lock the map viewport bounds to the North Indian Ocean (5°N–30°N, 45°E–105°E).
  - Implement a 15-step vertical slider component (0m to 1000m).
  - Implement color-mapping (e.g., standard oceanographic colorbars) for temperature slices.
  - Create a side-panel chart (e.g., using Chart.js or Recharts) to plot the vertical profile when a user clicks the map.
- **Step-by-Step Implementation Plan:**

1. Scaffold the React app and set up routing.
2. Build the base Leaflet map and restrict the panning bounds.
3. Create the API service file to fetch data from Member 3's backend.
4. Build the interactive depth slider state.
5. Build the vertical profile charting component.
6. Integrate the TCHP/D26 metric display on the dashboard.

- **Expected Code/Components:** MapViewer.jsx, DepthSlider.jsx, ProfileChart.jsx, apiService.js.
- **Expected Tests:** Component rendering tests (React Testing Library) ensuring the map

and sliders mount correctly.
- **Validation Criteria:** Changing the depth slider dynamically updates the map overlay; clicking the map successfully fetches and displays the vertical line chart.
- **Definition of Done:** The dashboard provides a lag-free visual interface to all 15 depths and cleanly handles API loading states.
- **Deliverables:** The completed React application source code and deployment build.
- **Handoff Procedure:** Merge the frontend code into the main branch and host it via a service like Vercel or Netlify for team access.
- **Failure Cases:** The backend goes down or returns 500 errors (mitigation: display a user-friendly "Service Unavailable" toast notification instead of crashing the UI).
- **How to Verify Independently:** Open the application in a browser, click a coordinate in the Bay of Bengal, and visually verify the chart renders a physically plausible temperature curve (warm surface, cold deep).
- **Exact Interface Contract:** Sends HTTP GET requests matching the OpenAPI schema. Expects JSON arrays in return.

Section 6: Member 5

- **Role:** Evaluation, Derived Metrics & Documentation Lead.
- **Objective:** Prove the scientific validity of the final model and compute disaster-management metrics.
- **Exact Responsibility:** Evaluate Kartik's final model against the 2021–2023 INCOIS Gridded ARGO test set, generate depth-wise RMSE/MAE statistics, implement the D26 Isotherm mathematical derivation, and compile the final repository README and architecture documentation.
- **Why this responsibility exists:** A model is useless without proven external validation. The problem statement explicitly requires tracking Tropical Cyclone Heat Potential (TCHP), which necessitates the D26 calculation.
- **Inputs Received:** The final model outputs (from Kartik) and the INCOIS ARGO test dataset.
- **Exact Input Format:** Predicted (B, 5, 15, 101, 241) tensors and ARGO ground truth arrays.
- **Expected Outputs:** Final scientific metrics, downstream calculation scripts, and presentation-ready documentation.
- **Exact Output Format:** Depth-wise evaluation tables, R^2 plots, the d26_calculator.py module, and README.md.
- **Files/Directories Owned:** src/evaluation/*, docs/*.
- **Files Allowed to Modify:** Evaluation scripts, markdown documentation, plotting utilities.
- **Files Must Not Modify:** Model training logic or the core dataset preprocessing.
- **Dependencies on Other Members:** Requires the final model outputs from Kartik and the backend integration from Member 3 for the D26 module.
- **What Others Depend On:** Member 3 requires the D26 Python module to serve the metric to the frontend. The team requires the metrics for the final pitch.
- **Detailed Tasks:**
  - Write the downstream calculation for D_{26}: scan the 15-depth profile, find where

temperature crosses 26°C, and linearly interpolate the exact depth.
    - Evaluate ConvFormer against the independent ARGO data to assess real-world generalization.
    - Calculate Overall RMSE, Overall MAE, Depth-wise RMSE, and R^2.
    - Format all technical documentation (including the Architecture specification) into the final GitHub structure.
- **Step-by-Step Implementation Plan:**
1. Draft the d26_calculator.py script using NumPy interpolation logic.
2. Write unit tests for edge cases (e.g., surface temp < 26°C returning NaN).
3. Hand the D26 module to Member 3 for backend integration.
4. Run the evaluation suite on the 2021-2023 ARGO test set.
5. Generate the depth-wise performance plots (0m down to 1000m).
6. Finalize the repository documentation.
- **Expected Code/Components:** evaluate.py, d26_calculator.py, plot_metrics.py.
- **Expected Tests:** Verify D26 interpolation mathematics against known manual calculations (e.g., if 50m is 27°C and 75m is 25°C, D26 must equal 62.5m).
- **Validation Criteria:** Evaluation metrics strictly exclude land masses; D26 accurately identifies the thermocline boundary without throwing index errors.
- **Definition of Done:** The final GitHub repository contains a polished README, architecture documentation, and a complete metric report for the 2021-2023 test period.
- **Deliverables:** The evaluation report, the derived metric module, and the final repository documentation.
- **Handoff Procedure:** Merge the documentation and evaluation artifacts into the main branch prior to the submission deadline.
- **Failure Cases:** ARGO coordinates slightly misalign with the GLORYS 0.25° grid (mitigation: implement a nearest-neighbor mapping function strictly for the evaluation step).
- **How to Verify Independently:** Review the generated evaluation plots and cross-reference the D26 output array against a raw temperature profile to manually confirm the 26°C crossing depth.
- **Exact Interface Contract:** Consumes un-normalized (15, H, W) temperature slices. Returns a 2D (H, W) array of D26 depth values in meters.